

Parallel & Distributed Computing



PARALLEL ALGORITHMS



Dr. Muzamil Ahmed (Assistant Professor)
Namal University Mianwali

Lesson from Quran

2

تِلْكَ آيَاتُ الْكِتَابِ الْحَكِيمِ ﴿٢﴾

هُدًى وَرَحْمَةً لِّلْمُحْسِنِينَ ﴿٣﴾

These are verses of the
Wise Book.
Guidance and mercy for the
good-doers.

Quran 31:2-3

QuranicQuotes.com

Lecture Outline

3

- Greedy Algorithms and Parallel Computing
- Why Greedy Algorithms Are Sequential
- Sources of Parallelism in Greedy Algorithms
- Case Studies – Parallel Kruskal's, Parallel Activity Selection
- Dynamic Programming and Parallel Computing
- Dependency Graph in Dynamic Programming
- Parallel Backtracking – Algorithm & Challenges
- Parallel Branch and Bound



Greedy Algorithms and Parallel Computing

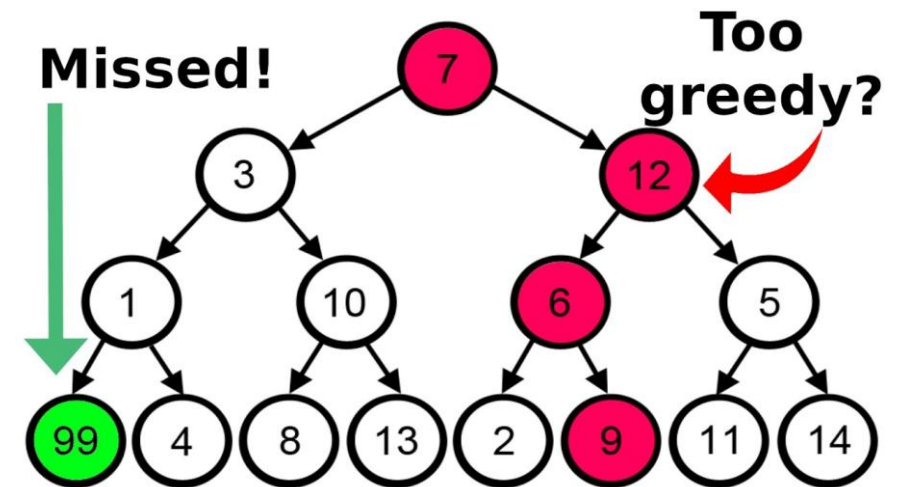
- A **greedy algorithm** constructs a solution incrementally by making the **best local decision** at each step.

- **Key Properties:**

- ▣ Local optimal choice
- ▣ No backtracking
- ▣ Fast and simple
- ▣ Often **sequential by design**

- **In Parallel Computing Context:**

- ▣ Greedy algorithms are **harder to parallelize** than divide-and-conquer
- ▣ Main challenge: **global decision dependency**



Why Greedy Algorithms Are Sequential

- Greedy algorithms rely on:
 - ▣ A **global best choice**
 - ▣ Previous decisions affecting future options
- **Example:**
 - ▣ In Dijkstra's algorithm:
 - ▣ You must select the node with the **minimum tentative distance**
 - ▣ This decision depends on all prior updates
- **Parallel Difficulty:**
 - ▣ Multiple processors competing to select the same “best” choice
 - ▣ Requires synchronization → performance overhead
- Naive parallelization often leads to poor speedup

Sources of Parallelism in Greedy Algorithms

- Despite limitations, parallelism exists in:
- **Candidate Evaluation**
 - ▣ Multiple choices evaluated simultaneously
- **Parallel Reduction**
 - ▣ Finding min/max using tree-based reduction
- **Parallel Sorting**
 - ▣ Many greedy algorithms start with sorting
 - Kruskal's Algorithm → Sort edges by weight
 - Activity Selection → Sort activities by finish time
 - Job Scheduling → Sort jobs by deadlines or profits
- **Relaxed Greedy Choices**
 - ▣ Allow near-optimal choices for better scalability
- Parallelize **what leads to the decision**, not the decision itself

General Parallel Greedy Pattern

- Sequential Greedy Structure:

```
while solution not complete:  
    find best candidate  
    add candidate to solution
```

- Parallel Greedy Structure:

```
while solution not complete:  
    parallel evaluate all candidates  
    best = parallel_reduce(candidates)  
    synchronize  
    update solution
```

- Parallelize computation, not commitment.

- Explanation:

- ▣ Evaluation is done in parallel
- ▣ Reduction selects the global best
- ▣ Synchronization ensures correctness
- ▣ Selection time reduces from $O(n)$ to $O(\log n)$

Parallel Reduction – Core Technique

8

- Parallel reduction is used to find:
 - ▣ Minimum
 - ▣ Maximum
 - ▣ Best-cost candidate
- **Pseudocode (Parallel Min):**

```
parallel for each processor  $P_i$ :  
    local_min[i] = min(elements assigned to  $P_i$ )  
global_min = reduce_min(local_min)
```
- **Explanation:**
 - ▣ Each processor works independently
 - ▣ Reduction tree combines results
 - ▣ Widely used in MST, Dijkstra, Scheduling
- Reduction requires synchronization at each level

Case Study – Parallel Activity Selection

9

- ❑ **Problem:**
- ❑ Select the maximum number of non-overlapping activities.
- ❑ **Sequential Greedy:**
- ❑ Sort by finish time
- ❑ Pick next compatible activity
- ❑ **Parallel Benefit:**
- ❑ Sorting dominates runtime
→ parallel sort gives high speedup
- ❑ Compatibility checks done concurrently

Parallel Version:

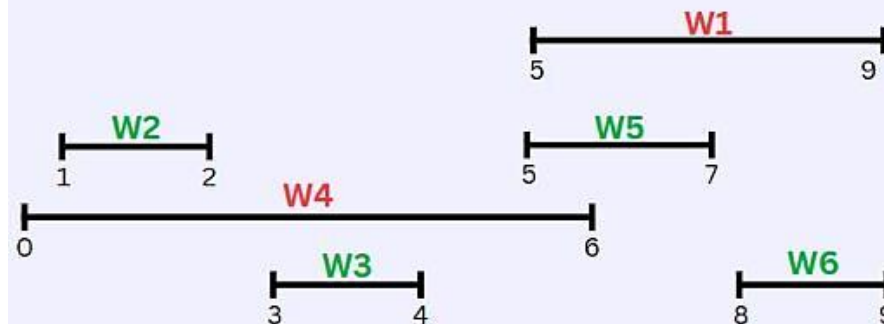
`parallel sort activities by finish time`
`parallel for each activity  $A_i$ :`

`check compatibility with previous selections`

`select compatible activities using prefix scan`

array = {{5,9},{1,2},{3,4},{0,6},{5,7},{8,9}}
 w1 w2 w3 w4 w5 w6

Wi = Selected Activity
Wi = Rejected Activity



Case Study – Parallel Kruskal's Algorithm

10

□ Sequential Kruskal:

- ▣ Sort edges by weight
- ▣ Add edge if it doesn't create a cycle

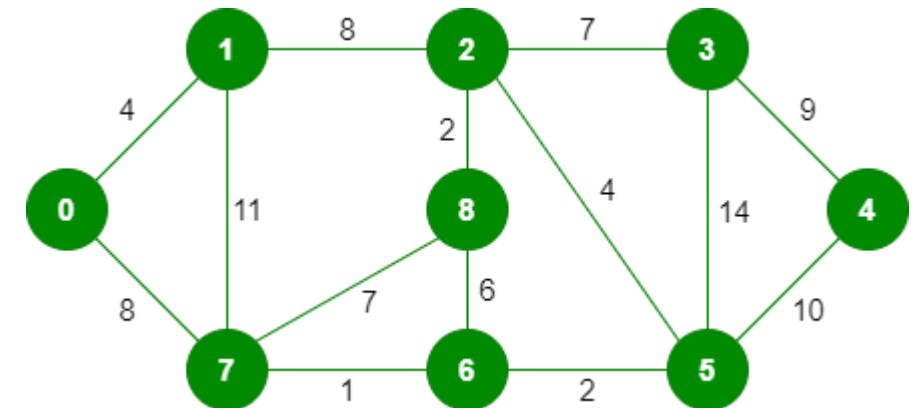
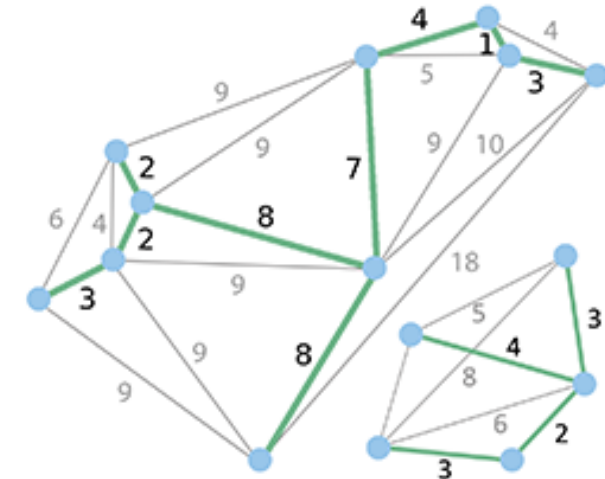
□ Parallel Kruskal:

parallel sort edges by weight
for each batch of edges in
parallel:

```
if find(u) != find(v):  
    add edge to MST  
    union(u, v)
```

□ Explanation:

- ▣ Sorting and edge checking are parallel
- ▣ Union-Find needs locks or atomic operations
- ▣ Best performance on large graphs



Parallel Dijkstra – Relaxed Greedy Approach

11

□ Problem:

- ▣ Selecting a single minimum node is sequential

□ Parallel Relaxation:

parallel relax edges of current frontier
select k smallest nodes instead of 1
process them in parallel

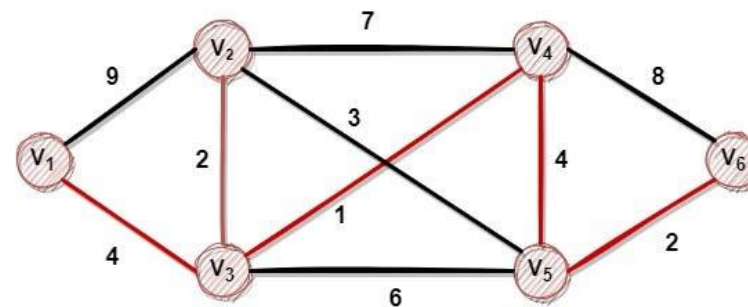
□ Explanation:

- ▣ Allows multiple near-minimum nodes
- ▣ Slight loss in strict greedy behavior
- ▣ Massive gain in parallelism

□ Used in:

- ▣ Large-scale graphs
- ▣ GPU-based shortest path algorithms

Dijkstra's Algorithm



Path: $V_1 - V_3 - V_4 - V_5 - V_6$

Total Cost: 11

	Distance	Previous Node	Visited
V_1	0	None	True
V_2	6	V_3	True
V_3	4	V_1	True
V_4	5	V_3	True
V_5	9	V_4	True
V_6	11	V_5	True

Dynamic Programming and Parallel Computing

12

- **Dynamic Programming (DP)** solves problems by:
 - ▣ Breaking them into **overlapping subproblems**
 - ▣ Solving each subproblem **once**
 - ▣ Storing results in a table (memoization)
- **In Parallel Computing:**
 - ▣ DP naturally exposes **structured parallelism**
 - ▣ Dependencies limit *when* subproblems can be computed

Why Dynamic Programming is Better than Greedy for Parallelism

13

Aspect	Greedy	Dynamic Programming
Dependencies	Strong	Structured
Parallelism	Limited	Moderate to High
Scalability	Medium	High
Synchronization	High	Predictable

- **Key Advantage:**
 - ▣ DP dependencies are **known and fixed**
 - ▣ Enables **wavefront parallelism**

Dependency Graph in Dynamic Programming

14

- DP problems form a **Directed Acyclic Graph (DAG)**

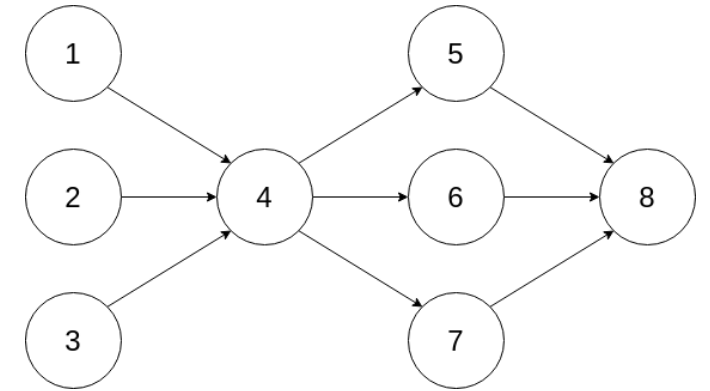
- **Example:**

- ▣ For 2D DP:

$$DP[i][j] = f(DP[i-1][j]DP[i][j-1])$$

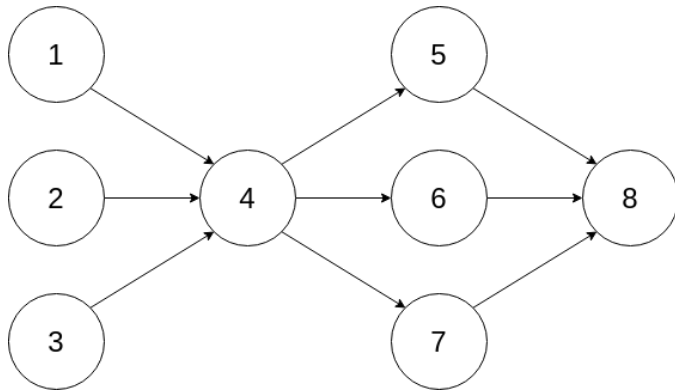
- **Observation:**

- ▣ DP cells on the **same diagonal** are independent
 - ▣ Can be computed **in parallel**
 - ▣ This is the foundation of parallel DP



Dependency Graph in Dynamic Programming

15



Node	Depends On	Can Run in Parallel?
1	—	Yes
2	—	Yes
3	—	Yes
4	1, 2, 3	No (waits for all)
5	4	Yes
6	4	Yes
7	4	Yes
8	5, 6, 7	No (waits for all)

Parallel DP Computation Pattern

16

□ Sequential DP:

```
for i = 1 to n:  
    for j = 1 to m:  
        DP[i][j] = f(DP[i-1][j], DP[i][j-1])
```

□ Parallel DP (Wavefront):

```
for d = 2 to n+m:  
    parallel for all (i, j) where i + j = d:  
        DP[i][j] = f(DP[i-1][j], DP[i][j-1])
```

□ Explanation:

- ▣ Each diagonal is computed in parallel
- ▣ Synchronization occurs between diagonals

Case Study – Parallel Matrix Chain Multiplication

17

□ DP Formula:

$$DP[i][j] = \min_{k=i}^{j-1} (DP[i][k] + DP[k+1][j] + cost)$$

□ Parallel Version:

for length = 2 to n:

parallel for all i:

j = i + length - 1

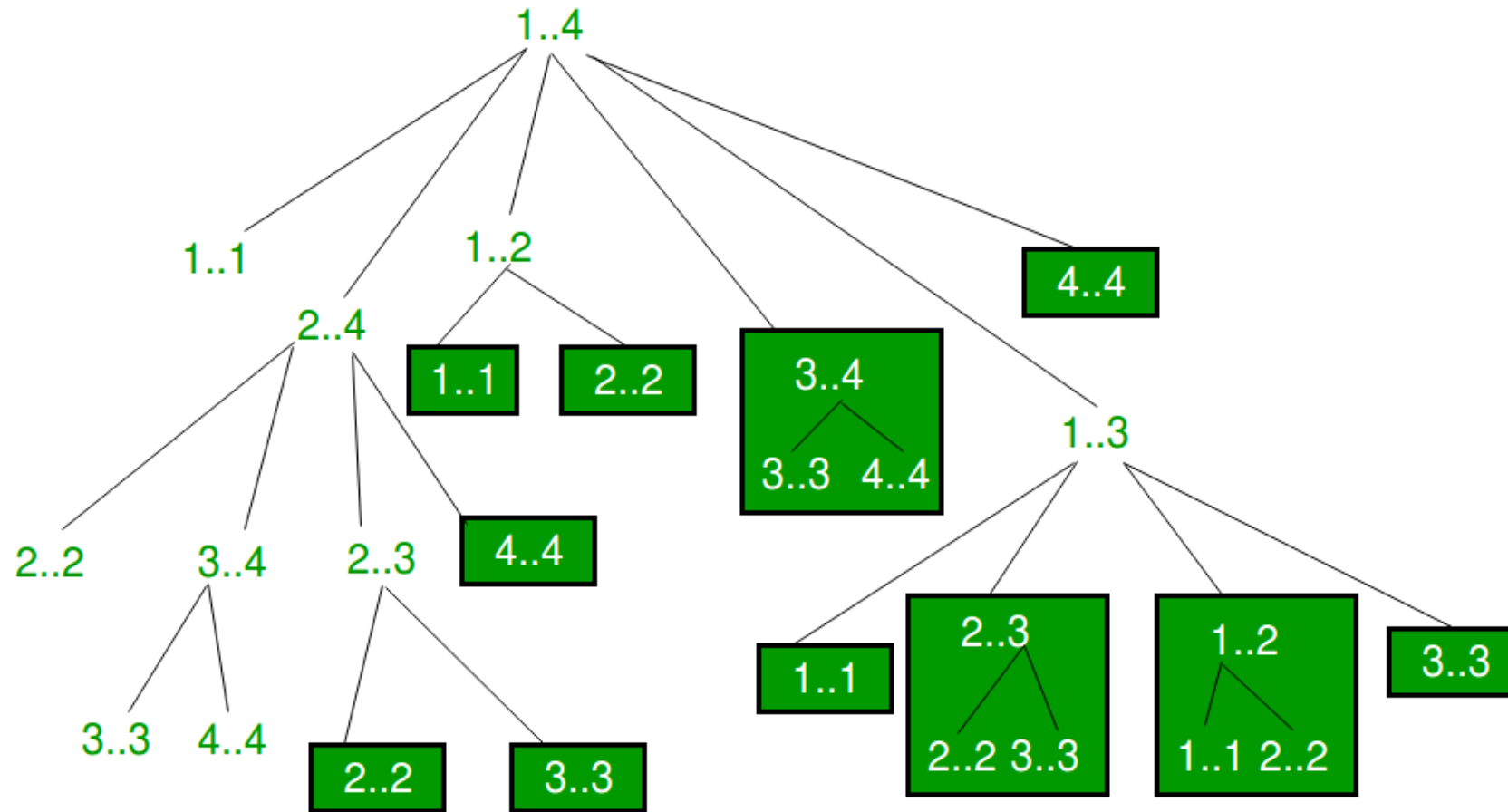
DP[i][j] = parallel_min over k

□ Explanation:

- ▣ Inner minimization is parallel
- ▣ Different (i, j) pairs computed concurrently

Case Study – Parallel Matrix Chain Multiplication

18



Case Study – Parallel Longest Common Subsequence

19

□ DP Formula:

if $X[i] == Y[j]$:

$DP[i][j] = DP[i-1][j-1] + 1$

else:

$DP[i][j] = \max(DP[i-1][j], DP[i][j-1])$

□ Parallel Approach:

- ▣ Compute table diagonally
- ▣ Each diagonal computed in parallel

□ Benefit:

- ▣ Excellent fit for shared-memory systems

		a	b	c	d	a	f
	0	0	0	0	0	0	0
a	0	1	1	1	1	1	1
c	0	1	1	2	2	2	2
b	0	1	2	2	2	2	2
c	0	1	2	3	3	3	3
f	0	1	2	3	3	3	4

Synchronization in Parallel DP

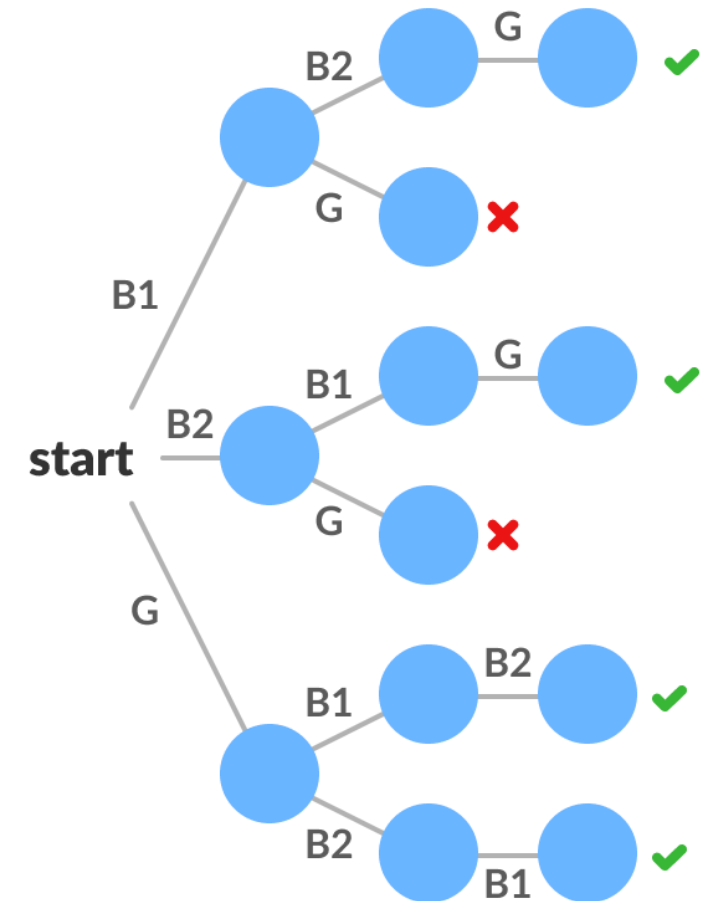
20

- **Why Synchronization is Needed:**
 - ▣ DP states depend on previous states
- **Strategy:**
 - ▣ Barrier synchronization between diagonals
 - ▣ Avoid fine-grained locks
- **Cost:**
 - ▣ Too many barriers reduce speedup
 - ▣ Optimal granularity is critical

Backtracking and Parallelism

21

- **Backtracking** is a systematic search technique that:
 - ▣ Explores all possible solutions
 - ▣ Abandons a path when it violates constraints
 - ▣ Uses **recursion and depth-first search**
- **Why Backtracking is Sequential:**
 - ▣ Decision tree explored one path at a time
 - ▣ Heavy dependency on previous choices
- **Parallel Opportunity:**
 - ▣ Different branches of the search tree are **independent**
 - ▣ Can be explored simultaneously by multiple processors



Parallel Backtracking – Algorithm & Challenges

22

□ Parallel Backtracking Pseudocode:

```
ParallelBacktrack(node) :  
    if solution found:  
        record solution  
    else:  
        parallel for each child of node:  
            if child is promising:  
                ParallelBacktrack(child)
```

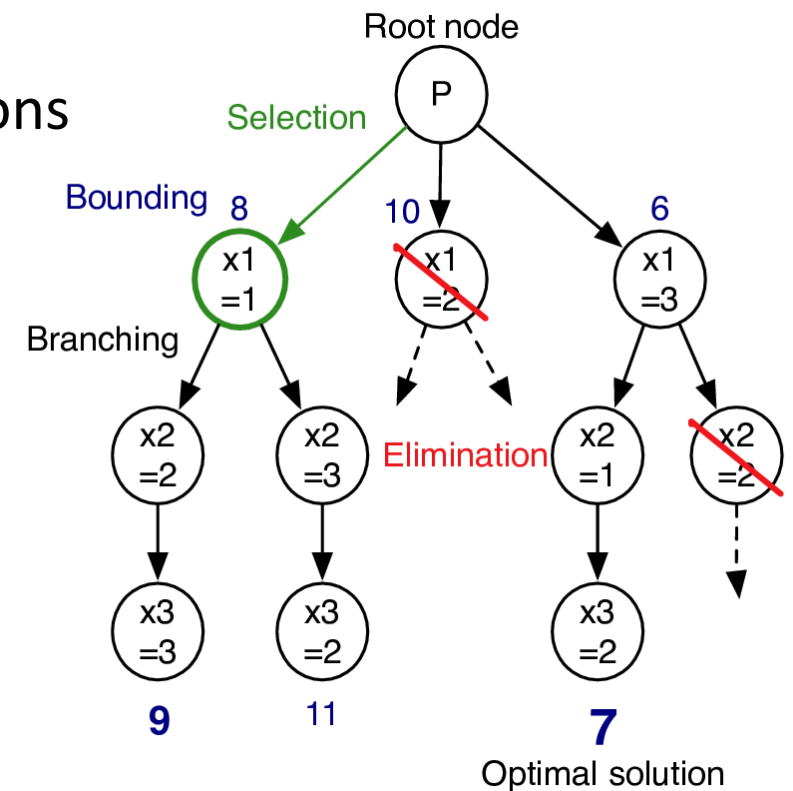
□ Parallel Challenges:

- ▣ Load imbalance (some branches deeper)
- ▣ Task granularity control
- ▣ Stack and memory management
- ▣ Early termination detection

Branch and Bound – Concept and Parallelism

23

- **Branch and Bound (B&B)** improves backtracking by:
 - ▣ Using a **bound** to prune search space
 - ▣ Ignoring branches that cannot produce better solutions
- **Components:**
 - ▣ Branching: Divide problem into subproblems
 - ▣ Bounding: Compute best possible solution
 - ▣ Pruning: Discard non-promising branches
- **Parallel Advantage:**
 - ▣ Independent subproblems
 - ▣ Faster pruning using shared bounds



Parallel Branch and Bound – Algorithm & Issues

24

□ Parallel B&B Pseudocode:

```
ParallelBnB(node, global_bound):  
    if bound(node) ≥ global_bound:  
        prune node  
    else:  
        parallel for each child:  
            ParallelBnB(child, global_bound)
```

□ Key Parallel Issues:

- ▣ Global bound update requires synchronization
- ▣ Frequent locking reduces performance
- ▣ Stale bounds cause redundant work

Parallel Algorithms – Unified Perspective

25

- Parallel algorithms aim to **reduce execution time** by exploiting **concurrency inherent in computational problems**, but the **degree and nature of parallelism** varies significantly across algorithmic paradigms.
- **Greedy Algorithms** offer limited parallelism due to strong sequential dependencies.
- Parallelism is mainly achieved through **parallel candidate evaluation, sorting, and reduction**, often trading strict optimality for scalability.
- **Dynamic Programming** is highly suitable for parallelization because of its **structured and predictable dependencies**.
- Techniques such as **wavefront (diagonal) parallelism** enable safe and efficient concurrent execution, making DP one of the most scalable parallel paradigms.

Parallel Algorithms – Unified Perspective

26

- **Backtracking Algorithms** naturally expose parallelism through **independent branches of the search tree**.
- However, challenges such as **load imbalance and irregular search depth** require dynamic task scheduling for effective performance.
- **Branch and Bound** enhances backtracking by introducing **global bounds** to aggressively prune the search space.
- While parallel exploration speeds up solution discovery, **synchronization of global bounds** becomes the main performance bottleneck.

Key Takeaway

27

- There is **no single parallelization strategy** that fits all algorithms.
- Effective parallel algorithm design requires:
 - ▣ Understanding **data dependencies**
 - ▣ Selecting appropriate **parallel decomposition strategies**
 - ▣ Balancing **parallelism, synchronization, and scalability**
 - ▣ **Efficient parallel algorithms do not merely run faster — they are carefully redesigned to respect correctness while maximizing concurrency.**

Thanks

28

**“A ship is safe
in harbor, but that’s
not what ships
are for.”**

JOHN A. SHEDD

